



Unidad 4a : Representación de datos georreferenciados

Introducción a Geopandas

Estructuras de datos

Entrada y salida de ficheros

Indexación y selección de datos

Proyecciones

Creando mapas

Operaciones con GeoDF

Introducción a Geopandas



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

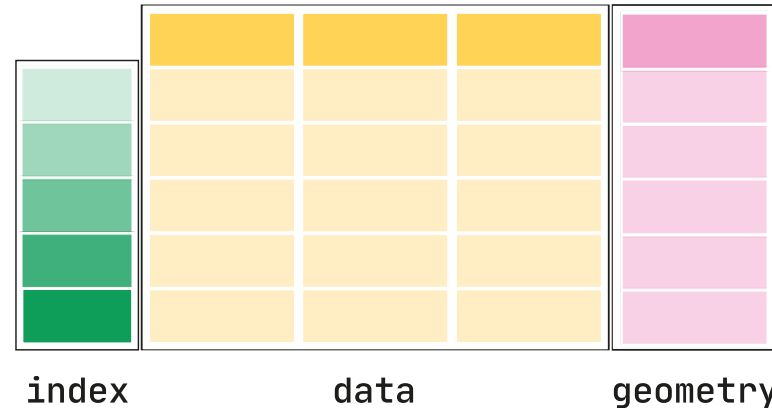
- ▶ La mayoría de información que se trata tiene un componente geoespacial
- ▶ En 1960 comienzan los Sistemas de Información Geográfica



Introducción a Geopandas



- ▶ ArcGis (ESRI) copa el mercado, pero hay muchas librerías de software libre
- ▶ Geopandas es una extensión de Pandas para tratar datos georreferenciados



- ▶ Dependencias:
 - ▶ pandas: manejo de tablas de datos
 - ▶ shapely: manejo de objetos geométricos planos
 - ▶ Fiona: I/O ficheros con datos geográficos (GDAL)
 - ▶ pyproj: transformaciones entre S.C.
 - ▶ packaging: manejo de paquetes de Python
 - ▶ rtree: Herramientas avanzadas de indexación espacial



- ▶ GeoPandas implementa dos estructuras de datos análogas a las Series y DataFrames:
 - ▶ GeoSeries y GeoDataFrame
- ▶ GeoSeries es un vector en el que cada elemento es un conjunto de una o más formas que corresponden a una observación



- ▶ GeoPandas tiene 3 clases básicas de objetos geométricos :
 - ▶ Puntos / Multi puntos
 - ▶ Líneas / Multi líneas
 - ▶ Polígonos / Multi polígonos
- ▶ La clase GeoSeries implementa:
 - ▶ Atributos
 - ▶ Métodos básicos
 - ▶ Testes de relaciones



- ▶ Atributos principales:
 - ▶ area: área de forma (unidades de proyección)
 - ▶ bounds: tupla de coordenadas máximas y mínimas en cada eje para cada forma
 - ▶ total_bounds: tupla de coordenadas máximas y mínimas en cada eje para toda la GeoSerie
 - ▶ geom_type: tipo de geometría
 - ▶ is_valid: prueba si las coordenadas hacen una forma que sea una forma geométrica razonable



- ▶ **Métodos básicos:**
 - ▶ `distance()`: devuelve una Serie con la distancia mínima de una entrada a otra
 - ▶ `centroid`: devuelve los centroides de la GeoSeries
 - ▶ `representative_point()`: devuelve una GeoSeries de puntos que se garantiza que están dentro de cada geometría. NO devuelve centroides.
 - ▶ `to_crs()`: cambia el sistema de coordenadas de referencia
 - ▶ `plot()`: plotea la GeoSeries



- ▶ Testes de relaciones:
 - ▶ `geom_almost_equals()`: tiene una forma parecida a otras (es interesante cuando los problemas de precisión de decimales hacen que las formas sean ligeramente diferentes)
 - ▶ `contains()`: si una forma contiene a otra
 - ▶ `intersects()`: si una forma interseca con otra



- ▶ Un GeoDataFrame es una estructura tabular que contiene GeoSeries
- ▶ Siempre hay una GeoSeries que contiene la geometría
- ▶ La columna geometría puede tener cualquier nombre: *gdf.geometry.name*
- ▶ Si hay varias columnas con geometría:
GeoDataFrame.set_geometry()
- ▶ Los atributos y métodos de GeoSeries se aplican a la columna geometría



- ▶ La información geográfica se almacena en dos tipos de ficheros:
 - ▶ Vectoriales: mediante primitivas
 - ▶ Raster: geoinformación a nivel de píxel
- ▶ Las coordenadas se suelen almacenar en sistema de coordenadas esféricas: longitud y latitud
- ▶ Los sistemas más utilizados son WGS84 (GPS) y ETRS89 (EU)
- ▶ Como son sistemas 3D se deben proyectar



Entrada y salida de ficheros

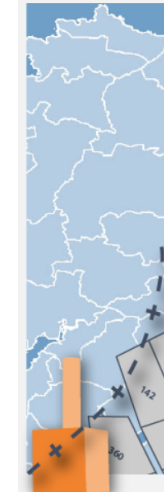


► Datos del IGN:

<https://centrodedescargas.cnig.es/CentroDescargas/index.jsp>

Líneas límite:

Nombre	Tamaño
 SHP_ETRS89	208 123 183
 SHP_WGS84	25 186 891
 Zona neutral entre España y Marruecos en Ceuta	16 866
 Zona neutral entre España y Marruecos en Melilla	4 167
 20190208MetadatosDivisionesAdministrativas.xml	38 329
 Leeme BDDAE.pdf	80 664



Límites municipales, provinciales y autonómicos

Descripción: recintos municipales y líneas límite (municipales, provinciales y autonómicos).

SGR: ETRS89 en la Península, Islas Baleares, Ceuta y Melilla, y WGS84 en las Islas Canarias. Coordenadas geográficas longitud y latitud.

Ud. descarga: toda España

Formato: shape (.shp)

Ver +

Metadatos

Descargar fichero atom



Descargar

► Datos ESRI España:

<https://hub.arcgis.com/search?collection=Dataset®ion=es§or=esri>

► Datos MITECO:

<https://www.miteco.gob.es/es/cartografia-y-sig/ide/descargas/>



- ▶ Los formatos de fichero más comunes son:
 - ▶ Shapefile: estándar de ArcGIS, puede estar formado por varios ficheros en un zip
 - ▶ KML: Formato de Google
 - ▶ GeoJSON: se utiliza mucho en webs
 - ▶ GPKG: formato abierto como contenedor de SQLite
 - ▶ CSV: fichero separado por , con IG
 - ▶ jpeg, TIFF, etc.



- ▶ Para leer un fichero se utiliza:
`geopandas.read_file()`
- ▶ Para ficheros zip:
 - ▶ Si es un único conjunto de ficheros se pone su nombre: `municipios = gpd.read_file('Municipios.zip')`
 - ▶ Si está en una carpeta:
`municipios = gpd.read_file('lineas_limite.zip!SHP_ETRS89/municipios')`
 - ▶ Si hay varios conjuntos de ficheros en la misma carpeta, se especifica el nombre del shp



- ▶ GeoPandas proporciona datasets:
 - ▶ naturalearth_cities: Localizaciones ciudades
 - ▶ naturalearth_lowres: Límites países
 - ▶ nybb: Límites NY
- ▶ Se puede filtrar la geometría por intersección:

```
gdf_mask = gpd.read_file(path+'naturalearth_lowres.zip')  
gdf = gpd.read_file(path+'naturalearth_cities.zip',  
mask=gdf_mask[gdf_mask.continent=="Africa"])
```



- ▶ También se puede filtrar por bbox:

```
bbox = (1031051.7879884212, 224272.49231459625,  
1047224.3104931959, 244317.30894023244)
```

```
gdf = gpd.read_file(path+'nybb.zip',bbox=bbox)
```

- ▶ Con el parámetro *rows*, se pueden filtrar una o varias filas: *rows=10*, *rows=slice(10,20)*
- ▶ Seleccionar o ignorar columnas:
 - ▶ *include_fields*, *ignore_fields*
- ▶ No cargar la geometría:
 - ▶ *ignore_geometry=True*



- ▶ Para escribir los datos en un fichero:

```
countries_gdf.to_file("countries.shp")
```

```
countries_gdf.to_file("countries.geojson",  
driver='GeoJSON')
```

```
countries_gdf.to_file("package.gpkg",  
layer='countries', driver="GPKG")
```

```
cities_gdf.to_file("package.gpkg",  
layer='cities', driver="GPKG")
```



- ▶ Se utilizan los mismos métodos que en Pandas:
 - ▶ `iloc`, `loc`
- ▶ Se añade el método `cx`, que permite seleccionar mediante `bbox`: `.cx[xmin:xmax,ymin:ymax]`

```
world = gpd.read_file(path+'naturalearth_lowres.zip')
```

```
southern_world = world.cx[:, :0] # xmin:xmax, ymin:ymax
```

Ejemplo4a1.py



- ▶ El sistema de coordenadas de referencia (CRS) indica la posición relativa de las posiciones en la tierra
- ▶ Los más utilizados son:
 - ▶ WGS84 o EPSG:4326
 - ▶ Zonas UTM norte: EPSG:32633
 - ▶ Zonas UTM sur: EPSG:32733
 - ▶ España: EPSG:32628 a EPSG:32631, siendo EPSG:25830 el más usado en la Península
- ▶ Las operaciones más importantes son: indicar la proyección y reprojectar



- ▶ Se debe indicar la proyección si no se ha hecho, en caso contrario, los datos no se dibujarán correctamente
- ▶ Los datos descargados de sitios oficiales indican su CRS
- ▶ Para saber el CRS: `GeoSeries.crs`
- ▶ Para indicar el CRS: `GeoSeries.set_crs(crs)`

```
my_geoseries = my_geoseries.set_crs("EPSG:4326")
```

```
my_geoseries = my_geoseries.set_crs(epsg=4326)
```

- ▶ Esto puede ocurrir cuando se convierte una lista de datos lon,lat a mano en una `GeoSeries`
- ▶ `set_crs` **asigna** un CRS a geometrías sin proyección (no transforma coordenadas), por lo que usar `set_crs` con el CRS incorrecto distorsiona la proyección



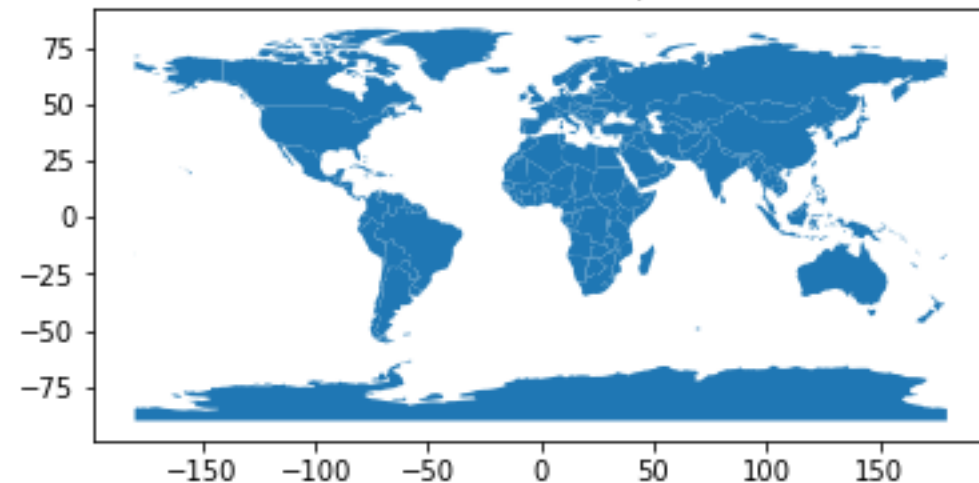
- ▶ La reproyección consiste en convertir la representación de las localizaciones de un CRS a otro
- ▶ Las proyecciones de datos sobre la tierra a un plano tendrán distorsiones: **`GeoDataFrame.to_crs()`**
- ▶ El método **`to_crs`** **reproyecta** (recalcula coordenadas)
- ▶ P.ej: para calcular centroides, áreas, etc. es necesario proyectar a un plano, se debe pasar de lat,lon a metros



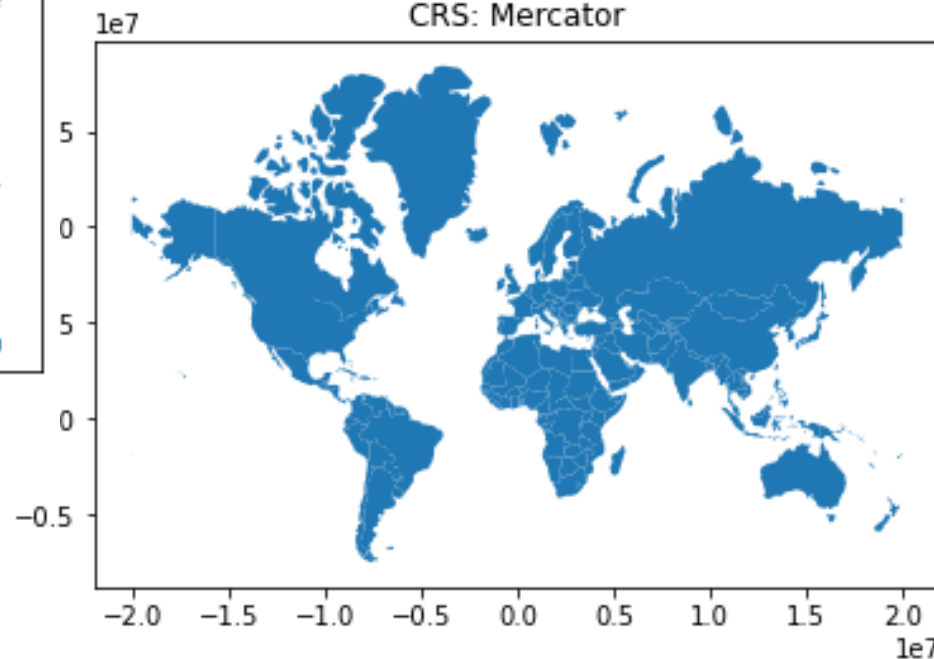
Proyecciones



CRS: WGS84 (lat/lon)



CRS: Mercator



- ▶ Ejemplo: Calcular área de las provincias españolas

- ▶ Para este tipo de cálculos funciona mejor una proyección cilíndrica:
+proj=cea

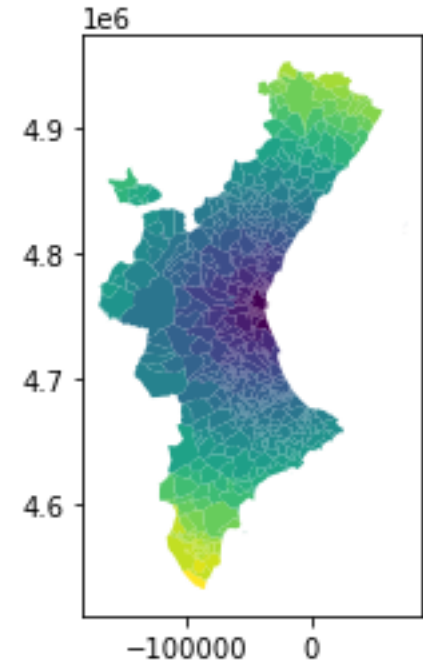
NAMEUNIT	area
Badajoz	21769.0
Cáceres	19865.0
Ciudad Real	19811.0
Zaragoza	17276.0
Cuenca	17139.0
Huesca	15638.0
León	15579.0
Toledo	15369.0

- ▶ Elegir un EPSG centrado en la proyección

```
provincias = gpd.read_file('Provincias_IGN.zip')  
provincias = provincias.to_crs("+proj=cea EPSG:2062")  
provincias['area'] = round(provincias.area/1000000,0)
```



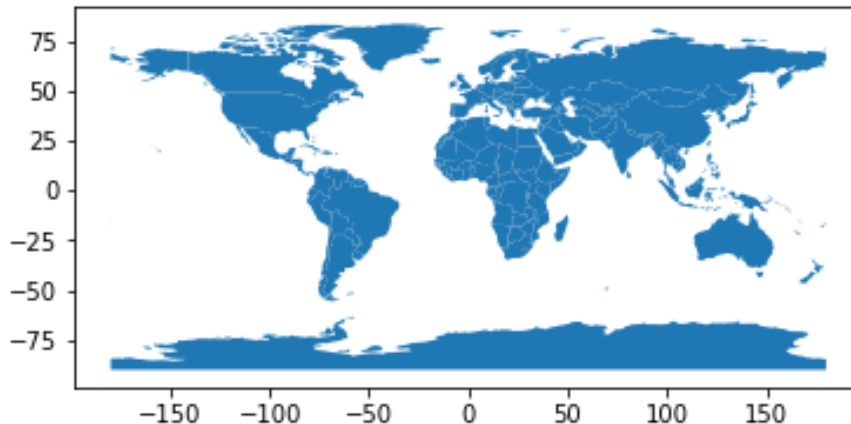
- ▶ Ejemplo: Calcular el centroide de cada municipio y la distancia a Valencia
- ▶ Seleccionar los municipios de la CV
 - ▶ CODNUT2='ES52'
- ▶ Ejemplo4a2.py



- ▶ GeoPandas crea una capa por encima de Matplotlib para dibujar mapas
- ▶ Método plot, al que se le pueden pasar parámetros
 - ▶ Ejemplo:

```
world = gpd.read_file(gpd.datasets.get_path('naturalearth_lowres'))
```

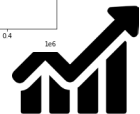
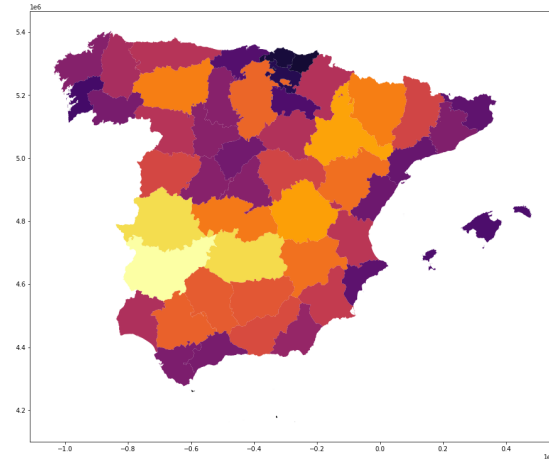
```
world.plot()
```



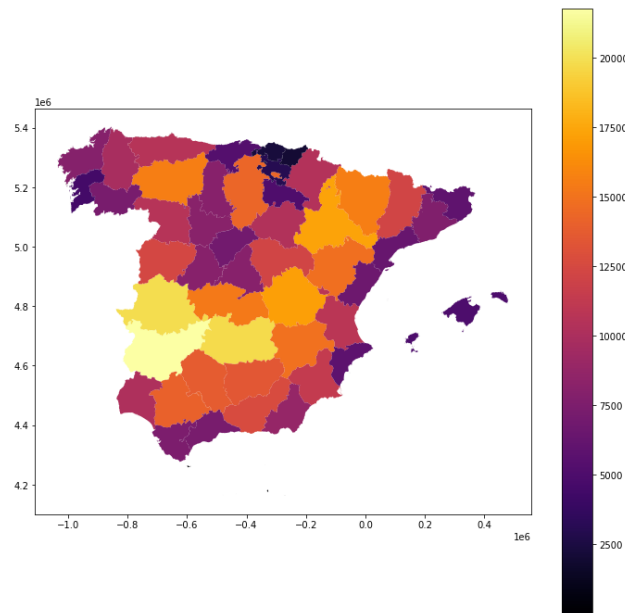
- ▶ Mapa de coropletas: mapa temático con las áreas sombreadas de diferentes colores en función del valor de una columna

```
provincias.plot(cmap='inferno', figsize=(15, 15), column='area')
```

<https://matplotlib.org/2.0.2/users/colormaps.html>

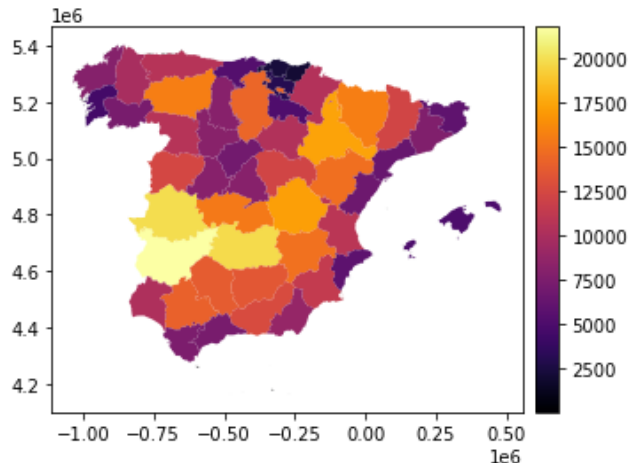


- Se puede añadir una leyenda: *legend=True*

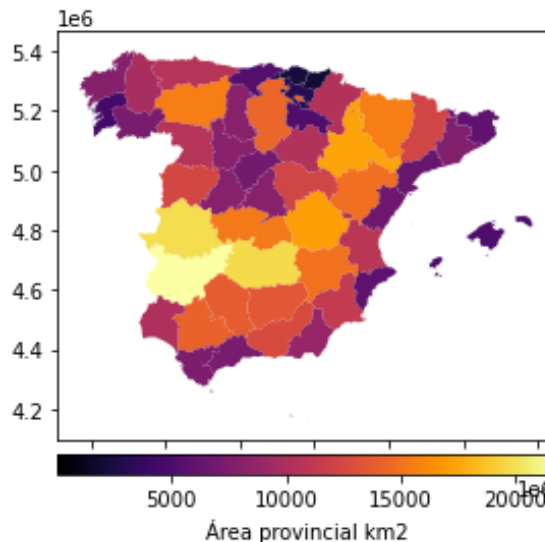


► Para que la leyenda se ajuste al mapa:

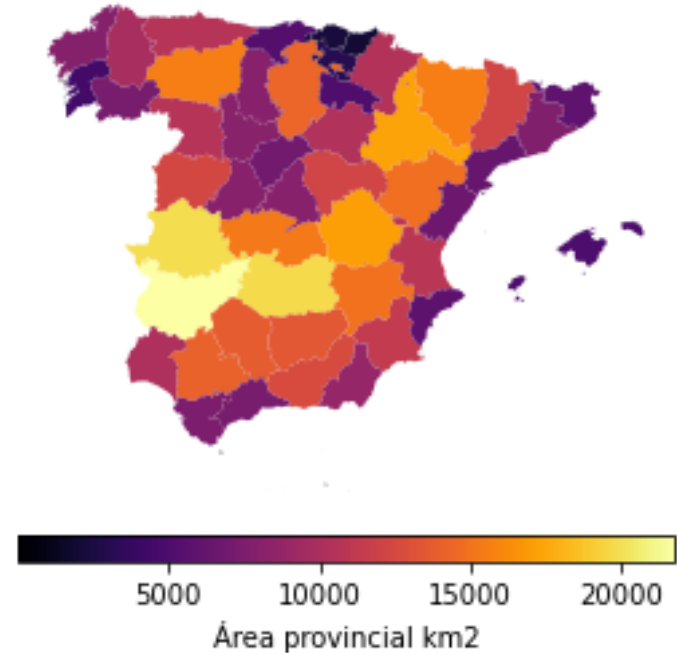
```
from mpl_toolkits.axes_grid1 import make_axes_locatable
fig, axis = plt.subplots(1, 1)
divider = make_axes_locatable(axis)
cax = divider.append_axes("right", size="5%", pad=0.1)
provincias.plot(ax = axis, cmap='inferno', column='area', legend=True,
cax=cax)
```



- ▶ Para que la leyenda aparezca en la parte inferior:
 - ▶ `legend_kwds={'label': "Área provincial km2", 'orientation': "horizontal"}`



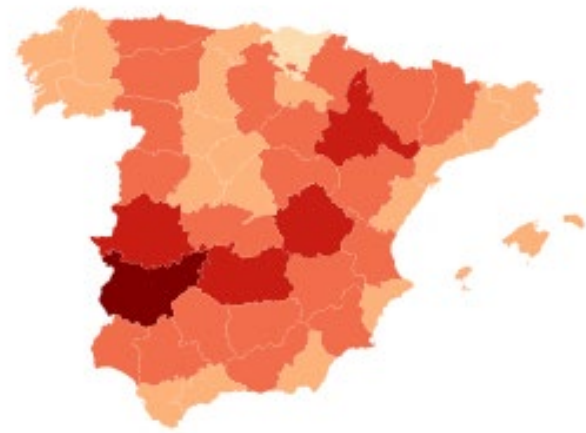
- **Importante:** La información de longitud y latitud, generalmente, no aportan nada. Si queremos eliminar los ejes: `axis.set_axis_off()`



- Los mapas de colores se pueden escalar mediante diferentes esquemas:

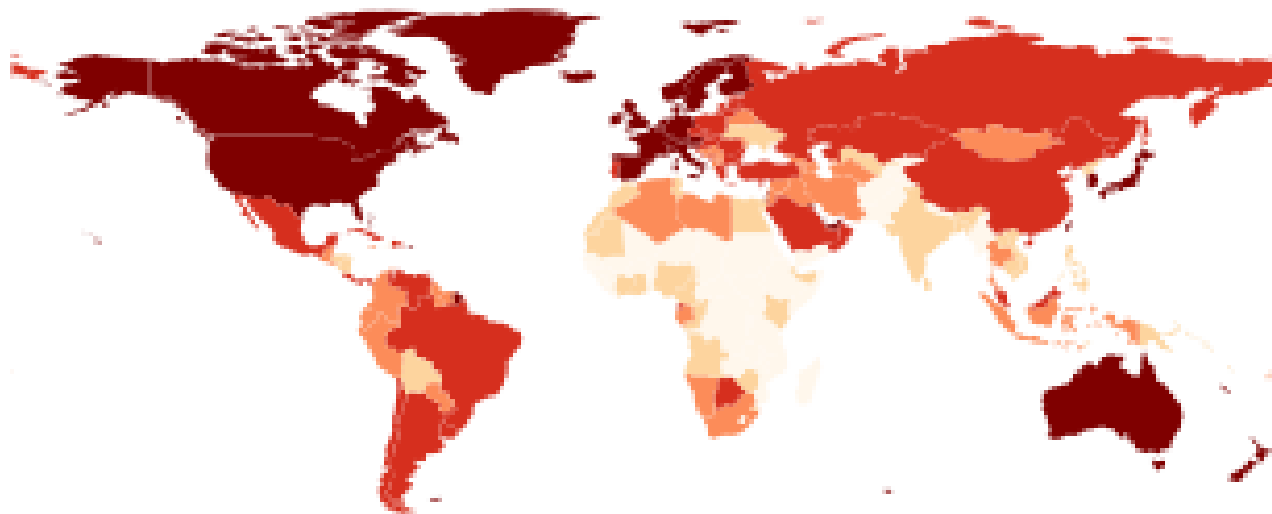
```
provincias.plot(cmap='OrRd', figsize=(12, 12), column='area',  
scheme='percentiles')
```

<https://github.com/pysal/mapclassify>



- ▶ Los mapas de colores se pueden escalar mediante diferentes esquemas:

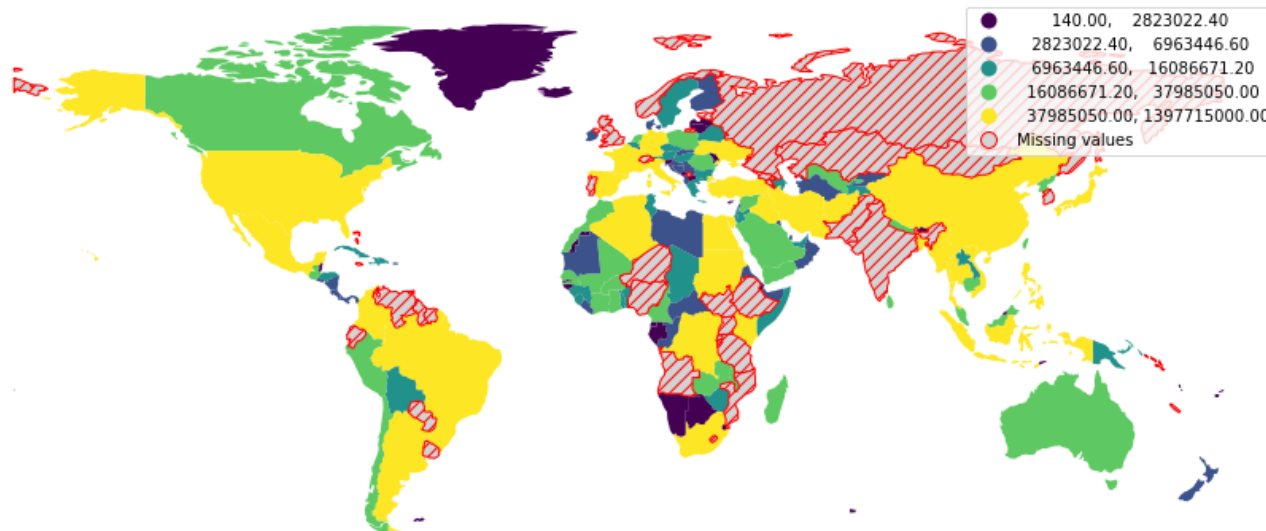
```
world.plot(ax=axis, column='gdp_per_cap', cmap='OrRd', scheme='quantiles')
```



- ▶ Cuando falte algún dato podemos indicar el formato de cómo se tiene que dibujar:

```
missing_kwds={"color": "lightgrey", "edgecolor": "red", "hatch": "///", "label": "Missing  
values"}
```

Ejemplo4a3.py



► Añadir etiquetas:

```
axis = provincias.boundary.plot(color='Black',  
linewidth=.4, figsize=(25,25))  
axis.set_axis_off()
```

```
provincias.apply(lambda x:  
axis.annotate(text=x.NAMEUNIT,  
xy=x.geometry.centroid.coords[0], ha='center',  
fontsize=12) if 'Territorio' not in x.NAMEUNIT  
else axis.annotate(text='',  
xy=x.geometry.centroid.coords[0], ha='center',  
fontsize=12), axis=1)
```

```
provincias.plot(ax=axis, cmap='Pastel2',  
figsize=(12, 12), column='NAMEUNIT')
```

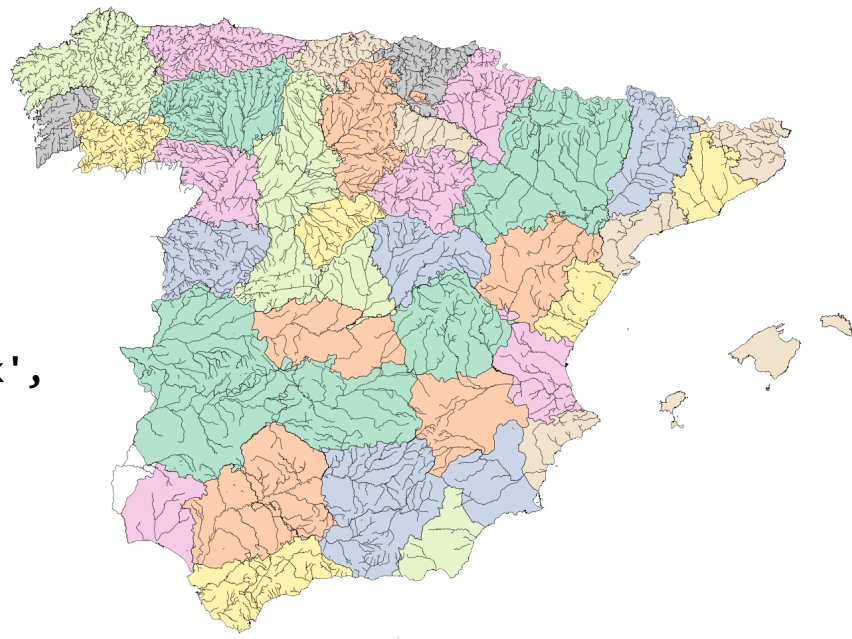


- ▶ Para crear mapas con diferentes capas se puede hacer de dos formas:
 - ▶ Combinando los mapas utilizando ejes, los CRS deben ser los mismos
 - ▶ Utilizando Matplotlib, se debe indicar:
 - ▶ `ax.set_aspect('equal')`
 - ▶ Con `zorder` se indica el orden de las capas a dibujar
 - ▶ Con las columnas de un GeoPandas se pueden dibujar cualquier tipo de gráfica de Pandas Matplotlib
 - ▶ `gdf.plot.bar()`



- Combinando los mapas utilizando ejes, los CRS deben ser los mismos

```
rios = gpd.read_file('Rios.zip')
rios = rios.to_crs(crs=3395)
axis = provincias.boundary.plot(color='Black',
linewidth=.4, figsize=(25,25))
provincias.plot(ax = axis, cmap='Pastel2')
rios[rios.TIPO_CORR=='RIO'].plot(ax = axis,
color='k', linewidth = 0.5)
```



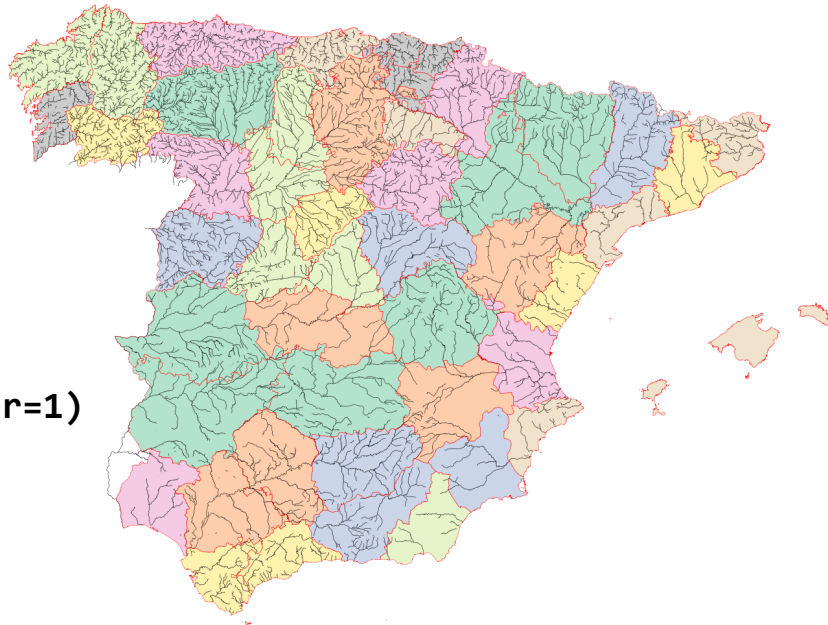
Rios

<https://www.miteco.gob.es/es/cartografia-y-sig/ide/descargas/agua/red-hidrografica.aspx>



► Utilizando Matplotlib:

```
fig, ax = plt.subplots(figsize=(25,25))
ax.set_aspect('equal')
ax.set_axis_off()
provincias.boundary.plot(ax=ax,color='Red',
linewidth=.4,zorder=3)
provincias.plot(ax = ax, cmap='Pastel2',zorder=1)
rios[rios.TIPO_CORR=='RIO'].plot(ax = ax,
color='k', linewidth = 0.5, zorder=2)
plt.show()
```



Ejemplo4a4.py

- ▶ Operaciones geométricas: buffer, boundary, centroid, convex_hull, envelope, simplify y unary_union
- ▶ Transformaciones: rotate, scale, translate, skew y affine_transformation

https://geopandas.org/en/stable/docs/user_guide/geometric_manipulations.html

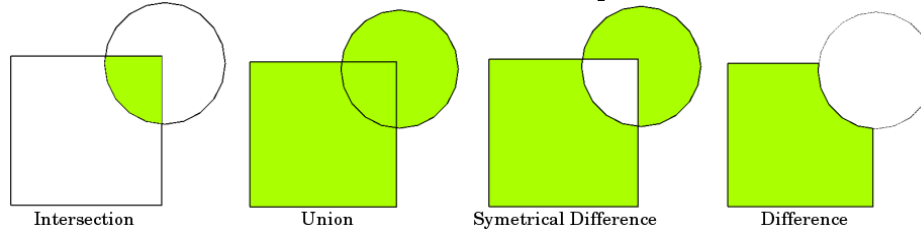


- ▶ `buffer()`: crear áreas de influencia alrededor de geometrías
- ▶ `clip()`: recortar geometrías por un polígono delimitador

```
valencia = municipios[municipios.NAMEUNIT ==  
'València'].to_crs(epsg=25830)  
buffer_val = valencia.buffer(50000) # 50 km  
provincias_cercanas =  
provincias.to_crs(epsg=25830).clip(buffer_val)
```



▶ Operaciones de conjunto:



- ▶ Se hacen con el método *overlay*:
- ▶ `res_union = df1.overlay(df2, how='union')`



- ▶ Agregación de GeoDatos: a veces tenemos la información geográfica más dividida de lo necesario y debemos agrupar
- ▶ El método *dissolve* es análogo a *groupby* en Pandas
 - ▶ disuelve todas las geometrías dentro de un grupo en una sola entidad geométrica
 - ▶ agrega todas las filas de datos en un grupo usando `groupby.aggregate`
 - ▶ combina los dos resultados
- ▶ Las operaciones posibles son:
'first', 'last', 'min', 'max', 'sum', 'mean', 'median', ...



- ▶ Ejemplo: tenemos el área por provincias y queremos mostrarla por CCAA

```
ccaa = provincias.dissolve(by='CODNUT2', aggfunc='sum')  
ccaa.plot(column = 'area', scheme='quantiles', cmap='YlOrRd')
```



- ▶ Hay dos formas de combinar GeoDF: unión de los atributos o unión espacial
- ▶ La unión de atributos es análogo al *merge* o *join* de Pandas
- ▶ En una unión espacial dos GeoSeries o GeoDF se combinan en función de su relación espacial



► Inicialización de los datos:

► Para unión de atributos

```
country_shapes = world[['geometry', 'iso_a3']]  
country_names = world[['name', 'iso_a3']]
```

► Para unión espacial

```
countries = world[['geometry', 'name']]  
countries = countries.rename(columns={'name': 'country'})
```

► Añadiendo GeoSeries

```
joined = pd.concat([world.geometry, cities.geometry])
```

► Añadiendo GeoDataFrames

```
europe = world[world.continent == 'Europe']  
asia = world[world.continent == 'Asia']  
eurasia = pd.concat([europe, asia])
```



- ▶ La unión de atributos se hace con el método `merge`
- ▶ Si queremos unir un GeoDF y un DF, el GeoDF debe ser el que llame al método
- ▶ Para añadir una columna con el nombre del país al GeoDF `country_shapes`:

```
country_shapes = country_shapes.merge(country_names, on='iso_a3')
```



- ▶ La unión espacial se hace entre dos GeoDF
- ▶ Hay dos métodos:
 - ▶ `gdf.sjoin()`: la unión se realiza en función de la operación indicada (*intersects*, *contains*, *within*, *touches*, *crosses*, *overlaps*)
 - ▶ `gdf.sjoin_nearest()`: la unión se realiza por proximidad, se puede indicar un radio.



- ▶ El método *sjoin* tiene dos parámetros:
 - ▶ *how*: *left*, *right* o *inner* (intersección), indica el índice a utilizar
 - ▶ *predicate*: *intersects*, *contains*, *within*, *touches*, *crosses*, *overlaps*, indica como se va a realizar la unión
- ▶ El método *sjoin.nearest* tiene tres parámetros:
 - ▶ *how*: igual que antes
 - ▶ *max_distance*: el radio máximo de búsqueda
 - ▶ *distance_col*: opcional, añade una columna con la distancia a la geometría más cercana



- Concatemos Europa y Asia y dibujamos:

```
eurasia = pd.concat([europe,asia])
```

```
eurasia.plot()
```

Ejemplo4a5.py



- ▶ Problema al dibujar Rusia: hacer un desplazamiento del mapa, Ejemplo4a6.py:

- ▶ <https://stackoverflow.com/questions/66833670/geopandas-map-centering-with-countries/>



- ▶ Partiendo del GeoDF de provincias:
 - ▶ Calcula el área de cada provincia
 - ▶ Crea un DF con el nombre y población de cada provincia
 - ▶ Añade estos datos al GeoDF
 - ▶ Calcula la densidad
 - ▶ Representa el mapa de la densidad de cada provincia, utiliza un mapa de color apropiado e incluye una leyenda
- ▶ Con estos mismos datos, representa un mapa con la densidad de cada CCAA



